

Chapter 21. Test-Driven Development (TDD)

Focusing on Unit Testing

Development testing

- Unit testing *→ 컴파일 테스트 / 보통 개발자 환경에서,*
 - individual program units or object classes are tested
 - unit testing should focus on testing the functionality of objects or methods.
- Component testing *→ 컴파일 연관은 테스트*
 - several individual units are integrated to create composite components.
 - component testing should focus on testing component interfaces.
- System testing
 - some or all of the components in a system are integrated and the system is tested as a whole.
 - system testing should focus on testing component interactions.

→ 테스트

Why Unit Testing?

- *Code isn't right if it's not tested.*
- Practical
 - Most programmers rely on testing
e.g., MS has 1 tester per developer
 - You could get work as a tester
- Divide-and-conquer approach
 - Split system into units
 - Debug unit individually
 - Narrow down places where bugs can be
 - Don't want to chase down bugs in other units

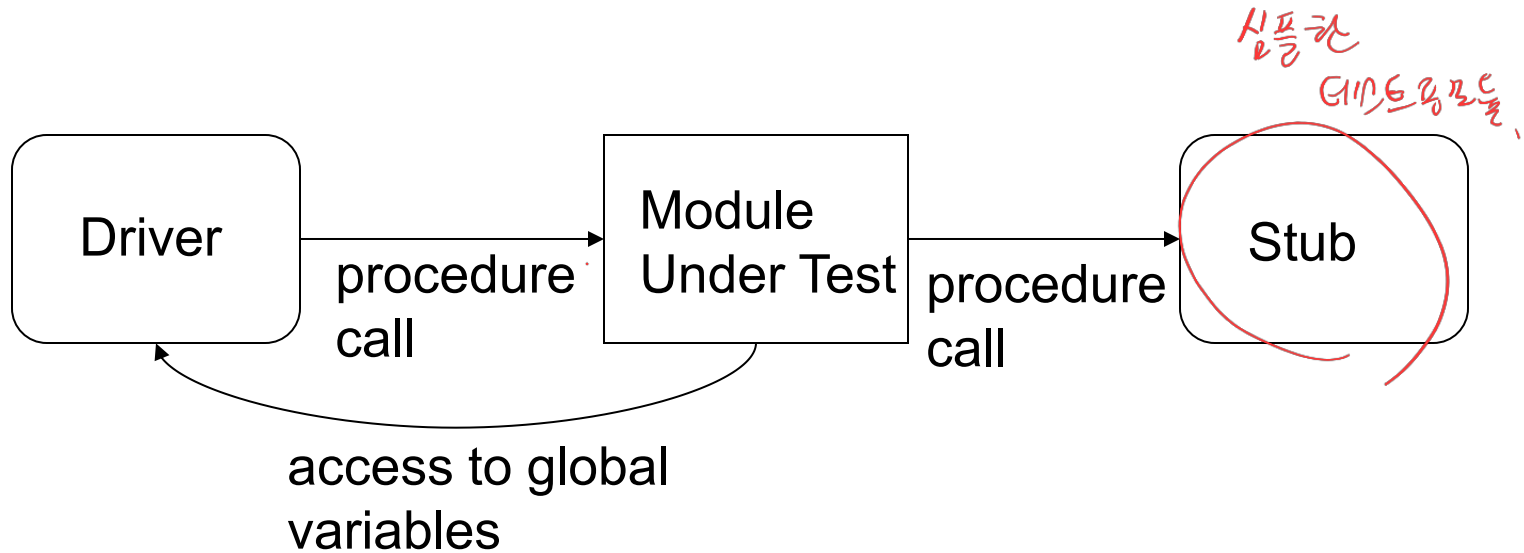
Unit Testing

- Involves testing a single isolated module
- Note that unit testing allows us to isolate the errors to a single module
 - we know that if we find an error during unit testing it is in the module we are testing
- Modules in a program are not isolated, they interact with each other. Possible interactions:
 - calling procedures in other modules
 - receiving procedure calls from other modules
 - sharing variables
- For unit testing we need to isolate the module we want to test, we do this using two things
 - drivers and stubs

Drivers and Stubs

- **Driver:** A program that calls the interface procedures of the module being tested and reports the results
 - A driver simulates a module that calls the module currently being tested
- **Stub:** A program that has the same interface as a module that is being used by the module being tested, but is simpler.
 - A stub simulates a module called by the module currently being tested

Drivers and Stubs



- Driver and Stub should have the same interface as the modules they replace
- Driver and Stub should be simpler than the modules they replace

Test suites

- Obviously you have to test your code to get it working in the first place
 - You can do *ad hoc* testing (testing whatever occurs to you at the moment), or
 - You can build a **test suite** (a thorough set of tests that can be run at any time)
- Disadvantages of writing a test suite
 - It's a lot of extra programming
 - *True*—but use of a good **test framework** can help quite a bit
 - You don't have time to do all that extra work
 - *False*—Experiments repeatedly show that test suites reduce debugging time more than the amount spent building the test suite
- Advantages of having a test suite
 - Your program will have fewer bugs
 - It will be a **lot** easier to maintain and modify your program
 - This is a *huge* win for programs that, unlike class assignments, get actual use!

Regression testing

- A type of software testing
 - verifies that software previously developed and tested still performs correctly after it was changed or interfaced with other software
 - changes may include software enhancements, patches, configuration changes, etc.
- Regression testing is required for a stable and maintainable code base

JUnit

- JUnit is a framework for writing unit tests
 - A **test case** is a single test of a single method
 - A **test suite** is a collection of test cases
- Unit testing is particularly important when agile approach is adopted
 - Code often has to be refactored to incorporate the changes
 - Unit testing helps ensure that the refactored code continues to work

JUnit

- Kent Beck developed the first xUnit automated test tool for Smalltalk in mid-90's
- Beck and Gamma (of design patterns Gang of Four) developed JUnit on a flight from Zurich to Washington, D.C.
- JUnit has become the standard tool for Test-Driven Development in Java
- JUnit test generators now part of many Java IDEs
- xUnit tools have since been developed for many other languages (Perl, C++, Python, Visual Basic, C#, ...)
 - JUnit uses Java's **reflection** capabilities
 - Java programs can examine their own code in run-time

Program to Test

```
public final class IMath {  
  
    /**  
     * Returns an integer approximation to the square root of x.  
     */  
    public static int isqrt(int x) {  
        int guess = 1;  
        while (guess * guess < x) {  
            guess++;  
        }  
        return guess;  
    }  
}
```

Conventional Test Case

```
/** A class to test the class IMath. */
public class IMathTestNoJUnit {
    /** Runs the tests. */
    public static void main(String[] args) {
        printTestResult(0);
        printTestResult(1);
        printTestResult(2);
        printTestResult(3);
        printTestResult(4);
        printTestResult(7);
        printTestResult(9);
        printTestResult(100);
    }
    private static void printTestResult(int arg) {
        System.out.print("isqrt(" + arg + ") ==> ");
        System.out.println(IMath.isqrt(arg));
    }
}
```

Conventional Test Output

```
lsqrt(0) ==> 1  
lsqrt(1) ==> 1  
lsqrt(2) ==> 2  
lsqrt(3) ==> 2  
lsqrt(4) ==> 2  
lsqrt(7) ==> 3  
lsqrt(9) ==> 3  
lsqrt(100) ==> 10
```

- What does this say about the code? Is it right?
- What's the problem with this kind of test output?

Solution?

- Automatic verification by testing program
 - Can write such a test program by yourself, or
 - Use a testing tool such as JUnit.
- JUnit
 - A simple, flexible, easy-to-use, open-source, and practical unit testing framework for Java.
 - Can deal with a large and extensive set of test cases.
 - Refer to www.junit.org.
 - NUnit, CppUnit, etc, ...

Testing with JUnit

```
import junit.framework.*;

/** A JUnit test class to test the class IMath. */
public class IMathTest extends TestCase {

    /** Tests isqrt. */
    public void testIsqrt() {
        assertEquals(0, IMath.isqrt(0)); // line 23
        assertEquals(1, IMath.isqrt(1));
        assertEquals(1, IMath.isqrt(2));
        assertEquals(1, IMath.isqrt(3));
        assertEquals(2, IMath.isqrt(4));
        assertEquals(2, IMath.isqrt(7));
        assertEquals(3, IMath.isqrt(9));
        assertEquals(10, IMath.isqrt(100));
    }
}
```

```
/** Returns the test suite for this test class. */
public static Test suite() {
    return new TestSuite(IMathTest.class);
}

/** Run the tests. */
public static void main(String[] args) {
    junit.textui.TestRunner.run(suite());
    // junit.swingui.TestRunner.run(suite());
}
}
```

Compilation and Output

```
$ javac IMath.java IMathTest.java
```

```
$ java IMathTest
```

```
.F
```

```
Time: 0.02
```

```
There was 1 failure:
```

```
1) testIsqrt(IMathTest)junit.framework.AssertionFailedError: expected:<0> but was:<1>
```

```
at IMathTest.testIsqrt(IMathTest.java:23)
```

```
at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
```

```
at sun.reflect.NativeMeth...
```

```
at sun.reflect.Delegating...
```

```
at IMathTest.main(IMathTest.java:17)
```

```
FAILURES!!!
```

```
Tests run: 1, Failures: 1, Errors: 0
```

Question: Is this better? Why?

Example: Manual Effort vs. JUnit

```
// Old way
void testMax() {
    int x = max(3, 7);
    if (x != 7) {
        System.out.println("max(3, 7) gives " + x);
    }
    x = max(3, -7);
    if (x != 3) {
        System.out.println("max(3, -7) gives " + x);
    }
}

public static void main(String[] args) {
    new MyClass().testMax();
}
```

```
// New way
@Test
void testMax() {
    assertEquals(7, max(3, 7));
    assertEquals(3, max(3, -7));
}
```

```
// Tested method
int max(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

Assertion Methods

Method	Description
<code>assertEquals(a,b)</code>	Test if a is equal to b
<code>assertTrue(a)</code>	Test if a is true
<code>assertFalse(a)</code>	Test if a is false
<code>assertNull(a)</code>	Test if a is null
<code>assertSame(a,b)</code>	Test if a and b refer to the identical object
<code>assertNotSame(a, b)</code>	Test if a and b do not refer to the identical object

- Static methods defined in `junit.framework.Assert`
- Variations taking string error messages

Warning: equals

- You can compare *primitives* with `==`
- Java has a method `x.equals(y)`, for comparing *objects*
 - This method works great for `Strings` and a few other Java classes
 - For objects of classes that *you* create, *you* have to define `equals`
- `assertEquals(expected, actual)` uses `==` or `equals`
- To define `equals` for your own objects, define *exactly* this method:
`public boolean equals(Object obj) { ... }`
 - The argument must be of type `Object`, which isn't what you want, so you must `cast` it to the correct type (say, `Person`):
 - ```
public boolean equals(Object something) {
 Person p = (Person)something;
 return this.name == p.name; // test whatever you like here
}
```

# Exercise

- Write a JUnit test class for testing

```
public class ForYou {
 /** Return the minimum of x and y. */
 public static int min(int x, int y) { ... }
}
```

# Exercise (Cont.)

```
public class ForYou {
 /** Return the minimum of x and y. */
 public static int min(int x, int y) { ... }
}
```

```
import junit.framework.*;
/** Test ForYou. */
public class ForYouTest extends TestCase {
 /** Test min. */
 public void testMin() {
 assertEquals(0, ForYou.min(100,0));
 assertEquals(0, ForYou.min(0,10));
 assertEquals(10, ForYou.min(10,10));

 assertEquals(-10, ForYou.min(10,-10));
 assertEquals(-100, ForYou.min(-10,-100));
 }
 // the rest as before ...
}
```

# JUnit – the Java's Unit Testing Framework

## :: Key JUnit Notions

### **Tested Class**

the class that is being tested.

### **Tested Method**

the method that is tested.

### **Test Case**

the testing of a class's method against some specified conditions.

### **Test Case Class**

a class performing the test cases.

### **Test Case Method**

a Test Case Class's method implementing a test case.

### **Test Suite**

a collection of test cases that can be tested in a single batch.

# JUnit – the Java's Unit Testing Framework

## :: Basics - JUnit 3

1. Create the class that you will want to test.
2. Build the test class – with the needed imports and extensions for JUnit.

We extend from `junit.framework.TestCase`.

We name the test methods with a prefix of 'test'.

3. Code the actual test cases.

We validate conditions using one of the several assert methods.

```
import junit.framework.*;

public class TestFailure extends TestCase { Test Case Class

 public void testSquareRootException() { Test Case Method
 try {
 SquareRoot.sqrt(-4); Tested Class and Method
 fail("Should raise an exception"); Assertion Statement
 }
 catch (Exception success) { ... }
 }
}
```

# JUnit – the Java’s Unit Testing Framework

## :: Basics – JUnit 4

Tests are identified by an **@Test** annotation

We no longer need to prefix our test methods with “test”.

```
import junit.framework.*;
import org.junit.Test;
public class TestAddition extends TestCase {

 private int x = 1;
 private int y = 2;
 @Test public void squareRootTest1()
 {
 int z = SquareRoot.sqrt(4);
 assertEquals(2, z);
 }
}
```



# Fixtures

- A fixture is just a some code you want to run **before/ after every test case**
- You get a fixture by overriding the method

```
protected void setUp() { ... }
protected void tearDown() { ... }
```
- The general rule for running a test is:

```
protected void runTest() {
 setUp(); <run the test>; tearDown();
}
```

  - so we can override setUp and/or tearDown, and that code will be run prior to or after every test case

# JUnit – the Java's Unit Testing Framework

## :: Basics – SetUp

- JUnit test runners automatically invoke the **setUp()** method before running **each test**.
- This method typically initializes fields, turns on logging, resets environment variables, and so forth.

```
protected void setUp()
{
 System.out.println("Before testing");
}
```

- In JUnit 4, the initialization method no longer needs to be called setUp().
- It just needs to be denoted with the **@Before** annotation.
- We can have multiple methods noted @Before, each running before testing.

```
@Before protected void initialize()
{
 System.out.println("Before testing");
}
```

# JUnit – the Java's Unit Testing Framework

## :: Basics – TearDown

- If we need **at the end of each test** to cleanup some flags we can use the JUnit's analog of *finalize()* – the ***tearDown()*** method. For example we can call the garbage collector there in case our tests consume large amount of memory.

```
protected void tearDown()
{
 System.out.println("After testing");
 System.gc();
}
```

- In JUnit 4, we can give it a more natural name and annotate it with **@After**.

```
@After protected void disposeObjects ()
{
 System.out.println("After testing");
 System.gc();
}
```

# JUnit – the Java's Unit Testing Framework

## :: Basics – Testing Exceptions

- Exception testing requires to wrap a try block around the code that throws the exception, then add a **fail()** statement at the end of the try block.

```
public void testDivisionByZero() {
 try {
 int n = 2 / 0;
 fail("Divided by zero!");
 } catch (ArithmeticException success) {
 assertNotNull(success.getMessage());
 }
}
```

- In JUnit 4, we can write the code that throws the exception and use an annotation to declare that the exception is expected:

```
@Test(expected=ArithmeticException.class) public void divideByZero()
{
 int n = 2 / 0;
}
```

# JUnit – the Java's Unit Testing Framework

## :: Basics – Timed Tests with JUnit 4

- Testing performance is one of the thorniest areas of unit testing.
- JUnit 4 doesn't solve this problem completely, but it does help.
- Tests can be annotated with a timeout parameter.
- If the test takes longer than the specified number of milliseconds to run, the test fails.

The following test fails if it takes longer than half a second to find all the elements in a document:

```
@Test(timeout=500) public void retrieveAllElementsInDocument()
{
 doc.query("//*");
}
```

# The structure of a test method

- A test method doesn't return a result
- If the tests run correctly, a test method does nothing
- If a test fails, it throws an `AssertionFailedError`
- The JUnit framework catches the error and deals with it; you don't have to do anything

# Example

```
public class PointTest extends TestCase {
 private Point p; // test fixture variable
 protected void setUp() { // initializes text fixture variables
 p = new Point(10, 10);
 }

 protected void tearDown() { } // clean up text fixture variables

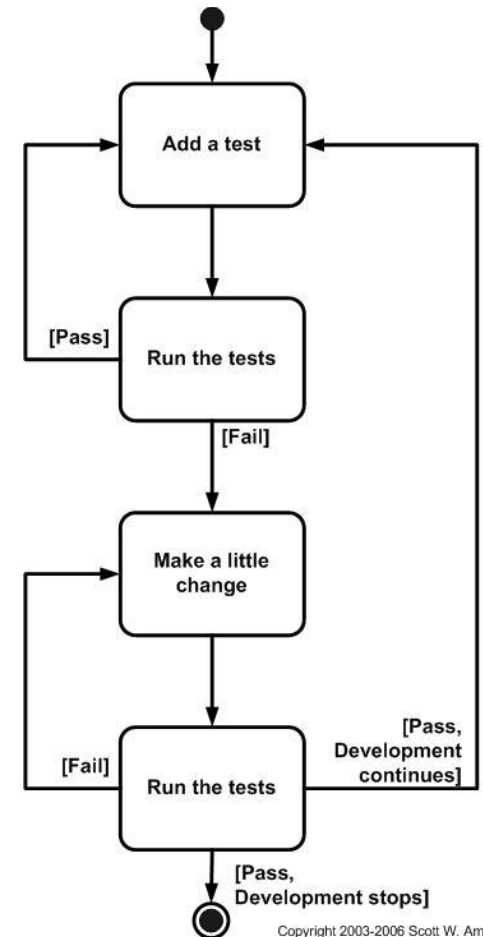
 public void testSetX() { // tests SetX
 p.setX(20);
 assertEquals(20, p.getX());
 }

 public void testSetY() { // tests SetY
 p.setY(30);
 assertEquals(30, p.getY());
 }
}
```

# Test-first development

test 케이스를 먼저 정하고, 그걸 통과할 수 있게끔 코드를 짜야한다

- Writing tests before code
  - clarifies the requirements to be implemented.
- Tests are written as programs
  - rather than data so that they can be executed automatically.
  - test includes a check that it has executed correctly
- All previous and new tests are automatically run
  - when new functionality is added
  - thus checking that the new functionality has not introduced errors





# Example: Counter class

- For the sake of example, we will create and test a trivial “counter” class
  - The constructor will create a counter and set it to zero
  - The **increment** method will add one to the counter and return the new value
  - The **decrement** method will subtract one from the counter and return the new value
- We write the test methods before we write the code
  - This has the advantages described earlier
  - However, we usually write the method **stubs** first, and let the IDE generate the test method stubs
- Don’t be alarmed if, in this simple example, the JUnit tests are more code than the class itself

# JUnit tests for Counter

```
public class CounterTest {
 Counter counter1; // declare a Counter here

 @Before
 void setUp() {
 counter1 = new Counter(); // initialize the Counter here
 }

 @Test
 public void testIncrement() {
 assertTrue(counter1.increment() == 1);
 assertTrue(counter1.increment() == 2);
 }

 @Test
 public void testDecrement() {
 assertTrue(counter1.decrement() == -1);
 }
}
```

- Note that each test begins with a *brand new* counter
- This means you don't have to worry about the order in which the tests are run

# The Counter class itself

```
public class Counter {
 int count = 0;

 public int increment() {
 return count += 1;
 }

 public int decrement() {
 return count -= 1;
 }

 public int getCount() {
 return count;
 }
}
```

- Is JUnit testing overkill for this little class?
- The Extreme Programming view is: *If it isn't tested, it doesn't work*
- You are not likely to have many classes this trivial in a real program, so writing JUnit tests for those few trivial classes is no big deal
- Often even XP programmers don't bother writing tests for *simple* getter methods such as `getCount()`
- We only used `assertTrue` in this example, but there are additional assert methods

# Writing a JUnit test class

- This page is really only for expensive setup, such as when you need to connect to a database to do your testing
  - If you wish, you can declare *one* method to be executed *just once*, when the class is first loaded

- **@BeforeClass**

```
public static void setUpClass() throws Exception {
 // one-time initialization code
}
```

- If you wish, you can declare *one* method to be executed *just once*, to do cleanup after all the tests have been completed

- **@AfterClass**

```
public static void tearDownClass() throws Exception {
 // one-time cleanup code
}
```

Handwritten notes in red ink:

setUpClass  
tearDownClass  
setUpClass  
tearDownClass

Handwritten note in red ink:

05/06.

# Examples with Stack Class

## Testing the Immutable Stack Class

```
public class Stack {
 public String toString()
 { // EFFECTS: Returns the String representation
 // of this Stack from the top to the bottom.
 StringBuffer buf = new StringBuffer ("{");
 for (int i = size-1; i >= 0; i--)
 {
 if (i < (size-1))
 buf.append(", ");
 buf.append (elements[i].toString());
 }
 buf.append ("}");
 return buf.toString();
 }
}
```

```
public boolean repOk() {
 if (elements == null) return false;
 if (size != elements.length) return false;
 for (int i = 0; i < size; i++) {
 if (elements[i] == null) return false;
 }
 return true;
}
```

# Examples with Stack Class

## Stack Test Class

- **Classes to import :**

```
import org.junit.After;
import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.assertEquals;
import junit.framework.JUnit4TestAdapter;
```

- **Setup operations :**

```
private Stack stack;
// setUp method using @Before syntax
// @Before methods are run before each test
@Before public void runBeforeEachTest()
{
 stack = new Stack();
}
```

- **Post-test operation :**

```
// tear-down method using @After
// @After methods are run after each test
@After public void runAfterEachTest()
{
 stack = null;
}
```

# Examples with Stack Class

## Stack Test Cases

```
@Test public void testToString()
{
 stack = stack.push (new Integer (1));
 stack = stack.push (new Integer (2));
 assertEquals ("{2, 1}", stack.toString());
}
```

**A problem with this test is that it actually combines four separate tests in one method**

**Without automation, large tests have the advantage of reducing costs of running many tests**

**With automation, small tests allow us to more easily identify failures ...**

Introduction to Software Testing (Ch 1), [www.introsoftwaretesting.com](http://www.introsoftwaretesting.com)

```
@Test public void testRepOk()
{
 boolean result = stack.repOk();
 assertEquals (true, result);
 stack = stack.push (new Integer (1));
 result = stack.repOk();
 assertEquals (true, result);
 stack = stack.pop();
 result = stack.repOk();
 assertEquals (true, result);
 stack = stack.push (new Integer (1));
 stack.top();
 result = stack.repOk();
 assertEquals (true, result);
}
```

## Stack Test Cases (2)

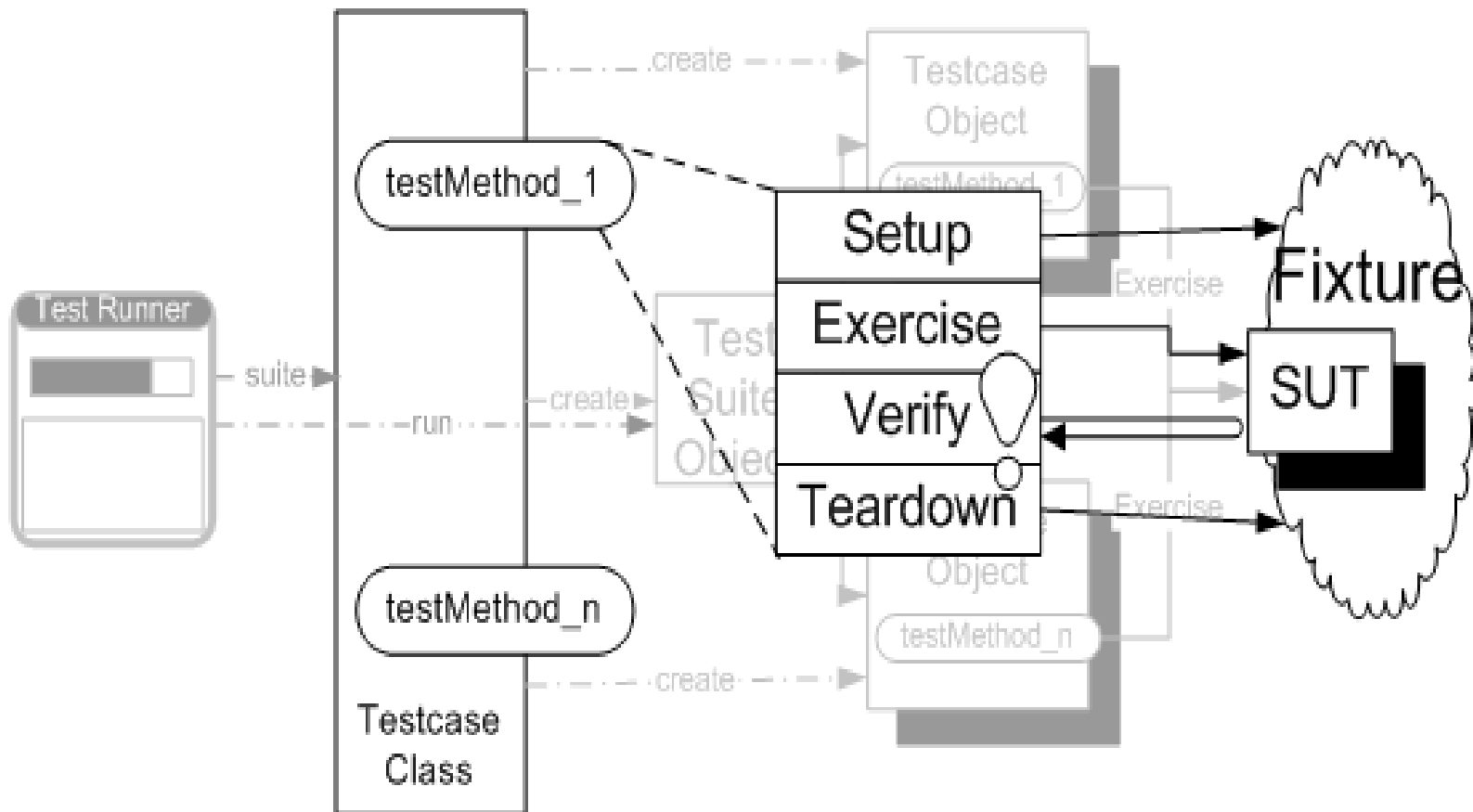
```
@Test public void testRepOkA()
{
 boolean result = stack.repOk();
 assertEquals (true, result);
}
```

```
@Test public void testRepOkB()
{
 stack = stack.push (new Integer (1));
 boolean result = stack.repOk();
 assertEquals (true, result);
}
```

```
@Test public void testRepOkC()
{
 stack = stack.push(new Integer(1));
 stack = stack.pop();
 boolean result = stack.repOk();
 assertEquals (true, result);
}
```

```
@Test public void testRepOkD()
{
 stack = stack.push (new Integer (1));
 stack.top();
 boolean result = stack.repOk();
 assertEquals (true, result);
}
```





# Viewing results in Eclipse

The screenshot shows the Eclipse IDE's test results window. At the top, a progress bar is red, indicating that not all tests passed. Below the bar, the summary shows 'Runs: 10/10', 'Errors: 2', and 'Failures: 0'. The list of tests includes 'testConstructor' through 'testSwap', with 'testIncompatibility' highlighted in yellow and marked with a red 'X' icon, indicating it failed. Callouts provide context: a green bar means all tests pass; a red bar means some failed; 'Ran 10 of the 10 tests' refers to the 'Runs' count; 'No tests failed, but...' refers to the 'Failures' count; 'Something unexpected happened in two tests' refers to the 'Errors' count; 'This test passed' points to a green checkmark; 'Something is wrong' points to a red 'X'; 'Depending on your preferences, this window might show only failed tests' points to the failed test; and 'This is how long the test took' points to the execution time.

Bar is green if *all* tests pass, red otherwise

Ran 10 of the 10 tests

No tests failed, but...

Something unexpected happened in two tests

Runs: 10/10 Errors: 2 Failures: 0

teamMaker.PairTest [Runner: JUnit 4] (0.027 s)

- testConstructor (0.001 s)
- testEquals (0.000 s)
- testToString (0.000 s)
- testGetStudent (0.000 s)
- testGetPartner (0.000 s)
- testReplaceStudent (0.000 s)
- testCrossover (0.000 s)
- testIncompatibility (0.025 s)
- testContains (0.001 s)
- testSwap (0.000 s)

This test passed

Something is wrong

Depending on your preferences, this window might show *only* failed tests

This is how long the test took

# Testing MakeLineItem()

```
public class SaleTest extends TestCase
{
 // ...

 // test the Sale.makeLineItem method
 public void testMakeLineItem()
 {
 // STEP 1: CREATE THE FIXTURE
 // -this is the object to test
 // -it is an idiom to name it 'fixture'
 // -it is often defined as an instance field rather than
 // a local variable
 Sale fixture = new Sale();

 // set up supporting objects for the test
 Money total = new Money(7.5);
 Money price = new Money(2.5);
 ItemID id = new ItemID(1);
 ProductDescription desc =
 new ProductDescription(id, price, "product 1");
 }
}
```

# Testing MakeLineItem() continued

```
// STEP 2: EXECUTE THE METHOD TO TEST
```

```
// NOTE: We write this code **imagining** there
// is a makeLineItem method. This act of imagination
// as we write the test tends to improve or clarify
// our understanding of the detailed interface to
// to the object. Thus TDD has the side-benefit of
// clarifying the detailed object design.
```

```
// test makeLineItem
```

```
sale.makeLineItem(desc, 1); // description and quantity
sale.makeLineItem(desc, 2);
```

```
// STEP 3: EVALUATE THE RESULTS
```

```
// there could be many assertTrue statements
// for a complex evaluation
```

```
// verify the total is 7.5
```

```
assertTrue(sale.getTotal().equals(total));
```

```
}
```

```
}
```

```

public class Sale
{
 private List<SalesLineItem> lineItems = new ArrayList()<SalesLineItem>;
 private Payment payment;

 private Date date = new Date();
 private boolean isComplete = false;

 public Money getBalance()
 {
 return payment.getAmount().minus(getTotal());
 }

 public void becomeComplete() { isComplete = true; }

 public boolean isComplete() { return isComplete; }

 public void makeLineItem(ProductDescription desc, int quantity)
 {
 lineItems.add(new SalesLineItem(desc, quantity));
 }

 public Money getTotal()
 {
 Money total = new Money(); Money subtotal = null;

 for (SalesLineItem lineItem : lineItems)
 {
 subtotal = lineItem.getSubtotal(); total.add(subtotal);
 }
 return total;
 }

 public void makePayment(Money cashTendered)
 {
 payment = new Payment(cashTendered);
 }
}

```